



Fundamentals in R

🔗 Certificate	R Fundamentals.pdf
📅 Completed	17/09/2026
# Hours	6
🔗 Link	https://coddy.tech/journeys/r/fundamentals
📍 Platform	Other
📅 Start	27/08/2026
⚙️ Status	In progress
🏷️ Tags	Tech



Introduction to R

Hello World

Comments

Variables and Data Types

Numeric Data Type

Integer Data Type

Character Data Type

Logical Data Type

Checking Data Types

Naming Conventions

Missing Values: NA

Operators

Arithmetic Operators

Integer Division and Modulo

Comparison Operators

Logical Operators (AND, OR, NOT)

Vectorized Logic

Decision Making

[If Statement](#)

[Else Statement](#)

[Nested If - Else](#)

[Basic IO](#)

[Print Output](#)

[Cat for Output](#)

[Output with Variables](#)

[Reading input with "readline"](#)

[Type Conversion Basics](#)

[substr\(text, start, stop\)](#)

[Project: Bill Split Calculator](#)

[Loops](#)

[For Loop](#)

[While Loop](#)

[Break](#)

[Next \(Continue\)](#)

[Sequence Generation \(seq, :\)](#)

[Nested Loops](#)

[Repeat](#)

[strrep\(\)](#)

[Functions](#)

[Declaring a Function](#)

[Function Arguments](#)

[Return values](#)

[Default Parameter Values](#)

[Vectors](#)

[Creating Vectors with c\(\)](#)

[Accessing Vector Elements](#)

[Modifying Vectors](#)

[Vector Operations](#)

[Named Vectors](#)

[Iterating over Vector Elements](#)

[Using seq_along\(\)](#)

[Iterating Characters \(strsplit\)](#)

[String Manipulation Functions](#)

[Vector Slicing and Indexing](#)

[Logical Indexing](#)

[Vector Arithmetic & Recycling](#)

[The %in% Operator](#)

[Lists in R](#)

[Creating Lists](#)

[Accessing List Elements](#)

[Modifying Lists](#)

[Named Lists](#)

Introduction to R

Application of R:

statistical computing data analysis

→ used by data scientists, statisticians, research

Hello World

`print()` to show words on the screen
→ anything inside quotation marks
is case sensitive (ex. capital letters)

```
print("Hello World")
```

```
[1] "Hello World"
```

Comments

notes that are written inside the code (R completely ignores them)

```
#This is a comment
```

Variables and Data Types

Numeric Data Type

Variables are created using the assignment operator `<-`, which is the standard way to assign values

Basic Syntax for creating a variable:

```
variable_name <- value
```

```
age <- 42
```

Once you create a variable, R remembers its value, and you can use the variable name anywhere you would use the actual value.

Numeric data type

used to store any number, whether it's a whole number like `42` or a decimal number `3.14`.

Integer Data Type

= specific integer data type for whole numbers (numbers without decimal points)

append the letter `L` to the number to modify a numeric data type to the integer data type

```
count <- 5L
```

```
age <- 25L
```

Character Data Type

= storing text information (compare "string")

to create a character variable you enclose your text in double quotes

```
name <- "Alice"
```

```
message <- "Welcome to R programming"
```

→ the quotes tell R that everything inside should be treated as text, not as code

Logical Data Type

The logical data type can only hold two possible values: `TRUE` and `FALSE`. These values represent the concepts of true and false, which are fundamental for controlling program flow and making comparisons.

Syntax of a logical variable

→ notice that both values are written in all capital letters without quotes

```
is_complete <- TRUE
```

```
has_error <- FALSE
```

Application: Decisionmaking, checking conditions, storing the results of comparisons

→ foundation for controlling what your code does in different situations

Checking Data Types

R provides the `class()` function to check the data type. The `class()` function takes a variable as input and returns a character string telling you the data type.

Simply pass your variable name inside the parentheses

```
my_variable <- 42
class(my_variable)

"numeric"
```

You can combine `class()` with `print()` to display the data type to the console, making it easy to verify what kind of data you're working with at any point in your program.

Naming Conventions

R has specific rules for valid variable names. A variable must start with a letter or a period not followed by a number, and can contain letters, numbers, periods, and underscores. however, it cannot start with a number or contain.

Popular styles are:

- `snake_case`
- `camelCase`

Missing Values: NA

In real-world data analysis, you'll often encounter situations where information is missing or unavailable. R provides a special value called `NA` to handle these cases.

`NA` stands for "Not Available" and represents a missing or undefined value. It's a special logical constant that serves as a placeholder when data is unknown or absent.

→ `NA` doesn't represent an actual value - it represents the absence of a value

Syntax of `NA`

```
missing_data <- NA
unknown_score <-
```

Operators

Arithmetic Operators

→ performing mathematical calculations in R

Operator	Operation	Example	Result
<code>+</code>	Addition	<code>5 + 3</code>	8
<code>-</code>	Subtraction	<code>10 - 4</code>	6
<code>*</code>	Multiplication	<code>6 * 7</code>	42
<code>/</code>	Division	<code>15 / 3</code>	5
<code>^</code>	Exponentiation	<code>2 ^ 3</code>	8

You can use these operators directly with numbers or with variables that contain numeric values:

```
price <- 25
quantity <- 4
total <- price * quantity
```

Integer Division and Modulo

= operators for working with whole numbers

Integer Division (`%/%`) divides two numbers and returns only the whole number part, discarding any remainder:

```
17 %/% 5
```

Return: 3 *not* 3.4

→ Think of it as asking “How many complete times does 5 fit into 17?”

Integer division helps when you need to distribute items into groups or convert units like minutes to hours.

Modulo (`%%`) returns the remainder after division

```
17 %% 5
```

Return: 2 *the remainder*

The modulo operator is commonly used to check if a number is even or odd

→ if `number %% 2` equals 0, the number is even

Comparison Operators

Unlike arithmetic operators that return numbers, comparison operators always return a logical value: either `TRUE` or `FALSE`

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	TRUE
<code>!=</code>	Not equal to	<code>5 != 3</code>	TRUE
<code>></code>	Greater than	<code>7 > 4</code>	TRUE
<code><</code>	Less than	<code>3 < 2</code>	FALSE
<code>>=</code>	Greater than or equal to	<code>5 >= 5</code>	TRUE
<code><=</code>	Less than or equal to	<code>4 <= 3</code>	FALSE

You can compare variables just like you compare numbers directly:

```
age <- 25
is_adult <- age >= 18
# TRUE
```

Be careful not to confuse `=` (assignment) with `==` (comparison). Using a single equals sign when you mean to compare is a common mistake that can lead to unexpected results.

Logical Operators (AND, OR, NOT)

Comparison operators let you check a single condition, but what if you need to check multiple conditions at once? Logical operators allow you to combine conditions and create more complex expressions.

The **AND** operator (`&&`) returns **TRUE** only when *both* conditions are *true*.

→ if either condition is false, the entire expression becomes false.

Think of it as requiring all conditions to pass.

```
age <- 25
has_license <- TRUE
```

```
can_drive <- age >= 18 && has_license
# TRUE
```

The **OR** operator (`||`) returns **TRUE** when *at least one* condition is true.

→ the **OR** operator only returns false when both conditions are false.

```
is_weekend <- TRUE
is_holiday <- FALSE
day_off <- is_weekend || is_holiday
#TRUE
```

A	B	A && B	A B
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE

A	B	A && B	A B
FALSE	FALSE	FALSE	FALSE

You can combine multiple conditions in a single expression using the operators together, building powerful logical checks for your programs.

The **NOT** operator (`!`) flips a logical value to the opposite. It turns **TRUE** into **FALSE** and **FALSE** into **TRUE** .

```
is_raining <- TRUE      is_closed <- FALSE
!is_raining             !is_closed
# FALSE                 # TRUE
```

This operator is useful when you want to check if something is not the case. For example instead of checking if a user is logged in, you might need to check if they are not logged in:

```
logged_in <- FALSE
if (!logged_in) {
  # Show login prompt
}
```

You can combine NOT with AND and OR operators to create more complex conditions:

```
has_ticket <- True
is_vip <- FALSE
# Can enter if they have a ticket AND are NOT a vip (regular entrance)
regular_entry <- has_ticket && !is-vip
# TRUE
```

Here is how NOT transforms logical values:

A	!A
TRUE	FALSE
FALSE	TRUE

Vectorized Logic

R has two versions of AND and OR operators.

- single values: `&& ||`
- vecorized values: `& |`

The key difference is that `&` and `|` compare values element by element

```
a <- c(TRUE, TRUE, FALSE, FALSE)
b <- c(TRUE, FALSE, TRUE, FALSE)
```

when working with multiple values:

```
a & b # TRUE FALSE FALSE FALSE
```

→ each position is compared independently

```
a | b # TRUE TRUE TRUE FALSE
```

This becomes powerful when combined with comparison operators:

```
scores <- c(85, 42, 91, 67, 55)
# Check which scores are between 50 and 80
scores >= 50 & scores <= 80
# FALSE, FALSE, FALSE, TRUE, TRUE
```

When you apply `!` to a vector of logical values, it flips each element individually.

```
passed <- c(TRUE, FALSE, TRUE, FALSE)
!passed #FALSE TRUE FALSE TRUE
```

Each `TRUE` becomes `FALSE`, and each `FALSE` becomes `TRUE`. This is particularly useful when you want to find elements that don't meet a certain condition.

```
temperatures <- c(18, 25, 30, 15, 22)
# Find temperatures that are NOT above 20
!(temperatures > 20) #TRUE FALSE FALSE TRUE FALSE
```

You can combine the vectorized NOT with `&` and `|` to build complex filters:

```
age <- c(16, 25, 18, 30, 14)
has_permission <- c(TRUE, TRUE, FALSE, TRUE, TRUE)
# Can participate if: age ≥ 18 OR (under 18 AND has permission)
can_participate <- ages >= 18 |(ages < 18 & has_permission)
# TRUE TRUE FALSE TRUE TRUE
```

Decision Making

If Statement

The `if` statement lets your code make decisions. An `if` statement runs a block of code only when a condition is `TRUE`:

```
temperature <- 35
if(temperature > 30){
  print("It's hot outside!")
}
```

When R encounters an `if` statement, it evaluates the condition inside the parenthesis.

- result is `TRUE`: the code inside the `{ }` executes

- result is `FALSE`: R skips that block entirely and continues with the rest of the program

You can use any expression that evaluates to a logical value as your condition, including the comparison and logical operators.

```
age <- 20
has_id <- TRUE
if(age >= 18 && has_id){
  print("Access granted")
}
```

Else Statement

With a basic `if` statement, code either runs or it doesn't. But what if you want to do something different when the condition is false? That's where `else` comes in.

→ The `else` block provides an alternative path when the `if` condition evaluates to `FALSE`:

```
temperature <- 18
if(temperature > 39){
  print("It's hot outside!")
} else{
  print("It's not too hot.")
}
```

This structure guarantees that exactly one of the two blocks will always run. Either the condition is true and the first block executes, or it's false and the `else` block executes:

Nested If - Else

Sometimes two options aren't enough. What if you need to check multiple conditions and handle each one differently? You can place an `if-else` statement inside another `if` or `else` block to create more complex decision paths.

```
score <- 85
if(score >= 90){
  print("Grade: A")
}else{
  if(score >= 80){
    print("Grade: B")
  }else{
    print("Grade: C or below")
  }
}
```

Cleaner way to write it, using `else` `if`:

```
if(score >=90){
  print("Grade: A")
}elseif(score >=80){
  print("Grade: B")
}
```

R first checks if the score is 90 or above. Since 85 doesn't meet that condition, it moves to the `else` block

```
}else{  
  print("Grade: C or below")  
}
```

Both approaches work the same way, but `else if` keeps your code flatter and easier to read. You can chain as many `else if` blocks as needed, and R will execute only the first block whose condition is true.

Basic IO

Print Output

The `print()` function displays a value to the console. It works with any data type:

```
print("Hello, World!")  
print(42)  
print(TRUE)
```

Each call to `print()` outputs on a new line. One thing to notice is that `print()` shows character strings with quotes around them. You can print the result of an expression directly:

```
print(5 + 3)    # Output: [1] 8  
print(10 > 5)   # Output: [1] TRUE
```

Cat for Output

The `cat()` function gives you a cleaner output compared to `print()`. While `print()` adds quotes around strings and shows index numbers, `cat()` displays text exactly as you want it to appear.

The name "cat" comes from "concatenate", meaning it can join multiple items together in a single output.

```
cat("Hello", "World")  
# Output: Hello World
```

Notices that `cat()` automatically adds a space between items. One difference from `print()` is that `cat()` doesn't automatically move to a new line. To create a line break, use `\n`:

```
cat("Line one\n")  
cat("Line two")  
#Line one  
#Line two
```

The `\n` or `\n` is called a newline character and has to be in quotation marks. You'll use `cat()` frequently when you want precise control over how your output looks.

Output with Variables

Let's see how to include variables in your output. This is essential for creating dynamic messages that change based on your data.

With `cat()`, you can mix text and variables by separating them with commas:

```
name <- "Alice"  
age <- 25  
  
cat("Name:", name, "\n")  
cat("Age:", age, "\n")  
#Name: Alice  
#Age: 25
```

→ you can mix multiple variables into the `cat()` and even include calculations:

```
price <- 10  
cat("Total cost:", price * 2, "\n")  
# Total cost: 20
```

Reading input with "readline"

Now it's time to make your programs interactive by reading input from the user. The `readline()` function lets you capture what the user types.

```
name <- readline()
cat("Hello," name, "\n")
```

When the code runs, R waits for the user to type something and press Enter. Whatever they type gets stored in the `name` variable.

`readline()` always returns a character string, even if the user types a number. If someone enters "25", it is stored as the text "25", not the number 25.

You can read multiple inputs by using `readline()` multiple times:

```
first <- readline()
second <- readline()
cat("First:", first, "\n")
cat("Second:", second, "\n")
```

Each `readline()` call waits for a separate line of input from the user.

When combining text with variables using `cat()`, each item is separated by a comma gets a space between it. Sometimes you need to join text without any space, for example, to attach punctuation directly to a word. That's where `paste0()` is useful. It concatenates strings with *no separator* between them:

```
city <- readline()
cat("Welcome to", paste0(city, "!"), "\n")
```

Here, `paste0(city, "!")` joins the value of `city` and `!"` together with no space, so if the user types "Paris", the output is: `Welcome to Paris!`

Type Conversion Basics

To convert a character string to a number, use `as.numeric()`:

```
input <- "42"
number <- as.numeric(input)

cat(number + 8, "\n") #Output: 50
```

You can convert in the opposite direction using `as.character()`.

substr(text, start, stop)

This function cuts a small string from a bigger string. Quotation marks are not counted.

```
substr("Hello World", 1, 5)
```

```
#Output:
```

```
[1] "Hello"
```

Project: Bill Split Calculator

```
con <- file( "stdin" , "r" ) :
```

- `"stdin"`: Refers to the *standard input stream* - where input comes from in a non-interactive script
- `"r"`: Mode `"r"` means "read" - you're opening this connection for reading data

```
bill <- as.numeric(suppressWarnings(readLines(con, n = 1)))
```

- `con`: The connection to `stdin` you opened earlier.
- `n = 1`: exactly one line from the input

```
close(con)
```

- closes the connection to `stdin`
- frees resources and signals that you're done reading from that stream
- some environments expect connections to be closed properly

```
cat("Bill: $", bill, "\n", sep = "")
```

- `sep = ""`:
 - by default, `cat()` inserts a space between each argument
 - `sep = ""` tells `cat()` to use an empty string as the separator
→ essentially a way to remove spaces

```
sprintf() :
```

- display monetary values with exactly 2 decimal places.

- for two decimal places: `%.2f`
- Syntax: `sprintf("%.2f", value)`

Loops

For Loop

Sometimes you need to repeat an action multiple times. Instead of writing the same code over and over, you can use a **for loop** to execute code repeatedly.

A for loop iterates through a sequence of values, running code inside the loop once for each value:

```
for (i in 1:3){  
  print(i)  
}
```

```
#Output:  
[1]  
[2]  
[3]
```

The variable `i` takes on each value in the sequence `1:3` (which means 1, 2, 3). For each value, the code inside the curly braces `{ }` runs once.

You can use any variable name and perform any operations inside the loop:

```
for (num in 1:4){  
  print(num *2)  
}
```

The loop multiplies each number by 2 before printing it.

While Loop

While a for loop iterates through a predefined sequence, a while loop keeps running as long as a condition remains `TRUE`. This is useful when you don't know in advance how many times you need to repeat something.

```
count <- 1  
while (count <= 3){.  
  print(count)
```

```
#Output:  
[1]  
[2]  
[3]
```

```
count <- count + 1  
}
```

→ IMPORTANT: You must update the variable inside the loop. Without `count <- count + 1`, the condition would always be `TRUE`, creating an infinite loop that never ends.

Break

Sometimes you need to exit a loop early, before it finishes all its iterations. The `break` statement lets you immediately stop a loop when a specific condition is met.

```
for (i in 1:10){  
  if (i == 5){  
    break  
  }  
  print(i)  
}
```

```
#Output:  
[1] 1  
[1] 2  
[1] 3  
[1] 4
```

This is especially useful with while loops when searching for something:

```
num <- 1  
while (TRUE){  
  if(num*num > 20){  
    break  
  }  
  num <- num + 1  
}  
print(num)
```

```
#Output  
[1] 5
```

Here, `while(TRUE)` would normally run forever, but `break` stops it once we find the first number whose exact square exceeds 20. This pattern is common when you need to search until a condition is satisfied.

Next (Continue)

While `break` exits a loop entirely, sometimes you only want to skip the current iteration and continue with the next one. The `next` statement does exactly that.

```
for (i in 1:5)
  if (i == 3){
    next
  }
  print(i)
}
```

#Output:

```
[1] 1
[1] 2
[1] 4
[1] 5
```

This is useful when you want to filter out certain values without stopping the entire loop. For example, printing only even numbers:

```
for(num in 1:6){
  if(num %% 2 != 0){
    next
  }
  print(num)
}
```

#Output

```
[1] 2
[1] 4
[1] 6
```

Here, odd numbers are skipped using `next`, and only even numbers get printed. The key difference from `break` is that the loop keeps running - it just skips specific iterations.

Sequence Generation (seq, :)

You have been using `1:5` to create sequences in your loops. This colon operator is quick and simple, but R also provides the `seq()` function for more control over your sequences.

The colon operator creates a sequence of consecutive integers:

```
print(1:5)
print(10:6)
```

```
[1] 1 2 3 4 5
[1] 10 9 8 7 6
```

Notice that the sequence can go in either direction. When the first number is larger, it counts down.

The `seq()` function gives you more flexibility. You can specify a step size using the `by` argument:

```
print(seq(2, 10, by = 2))  
#Output:  
[1] 2 4 6 8 10
```

You can also specify how many numbers you want using `length.out`:

```
print(seq(1, 10, length.out = 4))  
#Output:  
[1] 1 4 7 10
```

R automatically calculates the spacing to give you exactly 4 evenly spaced numbers. Both approaches work seamlessly in loops, letting you iterate through custom sequences with ease.

Nested Loops

You can place a loop inside another loop to create a nested loop. The inner loop runs completely for each iteration of the outer loop, making this useful when working with combinations or grid-like patterns.

```
for (i in 1:3){  
  for(j in 1:2){  
    cat("i = ", i ,  
      ", j =", "\n")  
  }  
}
```

```
#Output:  
i = 1 , j = 1  
i = 1 , j = 2  
i = 2 , j = 1  
i = 2 , j = 2  
i = 3 , j = 1  
i = 3 , j = 2
```

For each value of `i`, the inner loop cycles through all values of `j`. When `i` is 1, `j` goes from 1 to 2. Then `i` becomes 2, and `j` starts over from 1 to 2 again.

A practical example is creating a simple multiplication pattern:

```
for(row in 1:3){  
  for(col in 1:3){  
    cat(row * col,
```

```
#Output:  
1 2 3
```

```

    "" )
  }
  cat("\n")
}

```

```

2  4  6
3  6  9

```

When the first row was listed the `"\n"` jumps into the new row.

Repeat

The `repeat{}` function starts an endless loop in R. The function will run until it is explicitly ended using the `break` command. This is used if you are unsure how many entries will happen, until a certain key word that triggers the `break` command.

```

count <- 1

repeat {
  print(count)

  if (count == 5) {
    break
  }

  count <- count + 1
}

```

```

#Output:
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5

```

Beispiel: Dynamic input counter

```

repeat {
  input <- suppressWarnings(readLines(con, n = 1))
  if (input == "end") {
    break
  }
  num <- as.numeric(input)
  if (num > 0) {
    positive_count <- positive_count + 1
  } else if (num < 0) {
    negative_count <- negative_count + 1
  }
}

```

```
}  
}
```

If a user types "end" the break command is triggered. Until the amount of positive and negative numbers is counted.

strrep()

`strrep()` repeats a string a given amount of times.

```
strrep("*", 5)
```

#Output:

```
[1] "*****"
```

The string `"*"` is repeated `5` times.

Functions

Declaring a Function

So far, you've been writing code that runs from top to bottom. But what if you need to perform the same task multiple times in different places? Instead of copying and pasting code, you can create a function - a reusable block of code that you can call whenever you need it.

```
greet <- function(){  
  print("Hello!")  
}  
greet()
```

#Output:
[1] "Hello!"

We assign a function to the name `greet` using the assignment operator. The parentheses `()` after `function` are where parameters would go. The curly braces `{ }` contain the code that runs when the function is called. To use the function, you simply write its name followed by parentheses: `greet()`. This is called calling or invoking the function. You can call it as many times as you want. Functions help you organize your code, avoid repetition, and make your programs easier to read and maintain.

Function Arguments

Functions become much more powerful when they can accept input. Arguments (also called parameters) let you pass values into a function, making it flexible and reusable for different situations. To add arguments, place them inside the parentheses when defining the function.

```
greet <- function(name){  
  print(paste("Hello,",  
name))  
}  
greet("Alice")  
greet("Bob")
```

```
#Output:  
[1] "Hello, Alice"  
[1] "Hello, Bob"
```

You can define multiple arguments by separating them with commas:

```
add <- function(a, b) {  
  print(a + b)  
}  
  
add(5, 3)  
add(10, 20)
```

```
#Output:  
[1] 8  
[1] 30
```

When calling a function with multiple arguments, the values are matched to parameters in order. The first value goes to the first parameter, the second value to the second parameter, and so on.

Return values

So far, our functions have printed output directly. But often you want a function to compute a value and send it back so you can use it elsewhere in your code. This is where **return value** comes in.

```
add <- function(a,b){  
  return(a+b)  
}
```

```
#Output:  
[1] 8
```

```
result <- add(5,3)
print(result)
```

This makes functions more versatile:

```
double <- function(x){
  return(x*2)
}
print(double(5) + double
(3))
```

```
#Output:
[1] 16
```

In , if you don't use `return()` , the function automatically returns the last evaluated expression. However, using `return()` explicitly makes your code clearer and is especially useful when you want to exit a function early.

Default Parameter Values

Sometimes you want a function to have a "standard" behavior that can be customized when needed. Default parameter values let you define what a parameter should be if the caller doesn't provide it.

You set a default value by using `=` in the function definition:

```
greet <- function(name =
"Guest"){
  print(paste("Hello,",
name))
}
greet()
greet("Alice")
```

```
#Output:
[1] "Hello, Guest"
[1] "Hello, Alice"
```

When called without an argument, the function uses "Guest" as the default. When you provide a value, it overrides the default.

You can mix required and optional parameters. Parameters with defaults should come after those without:

```
calculate_total <- function(price, tax_r
ate = 0.1){
```

```
#Output:
[1] 110
```

```

    return(price + (price * tax_rate))
  }
print(calculate_total(100))
print(calculate_total(100, 0.2))

```

```
[1] 120
```

Vectors

Creating Vectors with c()

What if you need to store multiple related values together, like a list of temperatures or a set of scores? This is where vectors come in.

A vector is R's fundamental data structure for storing multiple values of the same type. You create vectors using the `c()` function, where "c" stands for "combine":

```

temperatures <- c(72, 75, 68, 80, 77)
print(temperatures)
#Output:
[1] 72 75 68 80 77

```

One important rule: all elements in a vector must be the same type. If you mix types, R will automatically convert them to a common type. You can check a vector's length using `length()`:

```

scores <- c(85, 92, 78, 95)
print(length(scores))
#Output:
[1] 4

```

Accessing Vector Elements

Now that you can create vectors, you need to know how to retrieve specific values from them. In R, you can access vector elements using square brackets `[]` with the position number of the element you want.

Unlike many programming languages, R uses 1-based indexing - the first element is at position 1, not 0:

```
fruits <- c("apple", "banana",  
"cherry", "date")  
print(fruits[1])  
print(fruits[3])
```

```
#Output:  
[1] "apple"  
[1] "cherry"
```

You can also access multiple elements at once by providing a vector of positions:

```
scores <- c(85, 92, 78, 9  
5, 88)  
print(scores[c(1, 3, 5)])
```

```
#Output:  
[1] 85 78 88
```

To access the last element of a vector, combine `length()` with indexing:

```
numbers <- c(10, 20, 30,  
40)  
print(numbers[length(num  
bers)])
```

```
#Output:  
[1] 40
```

Modifying Vectors

Let's explore how to change the values stored in a vector after it's been created. To modify a single element, use the assignment operator with the element's index:

```
scores <- c(85, 92, 78, 9  
5)  
scores[2] <- 88  
print(scores)
```

```
#Output:  
[1] 85 88 78 95
```

You can also modify multiple elements at once by providing a vector of positions:

```
temperatures <- c(70, 72,  
68, 75, 71)  
temperatures[c(1, 3)] <-
```

```
#Output:  
[1] 69 72 67 75 71
```

```
c(69, 67)
print(temperatures)
```

To add new elements to a vector you can assign a value to position beyond the current length, or use `c()` to combine vectors:

```
colors <- c("red", "blue")
colors[3] <- "green"
colors <- c(colors,
"yellow")
print(colors)
```

```
#Output:
[1] "red"    "blue"    "green"
"yellow"
```

Vector Operations

One of R's most powerful features is the ability to perform operations on entire vectors at once, without needing to loop through each element. This makes your code both cleaner and more efficient.

When you apply an arithmetic operator to a vector, R performs the operation on every element:

```
prices <- c(10, 25, 15, 30)
discounted <- prices * 0.9
print(discounted)
```

```
#Output:
[1] 9.0 22.5 13.5 27.0
```

You can also perform operations between two vectors of the same length. R pairs up corresponding elements:

```
quantities <- c(2, 1, 4, 3)
prices <- c(10, 25, 15, 30)
totals <- quantities * pr
```

```
[1] 20 25 60 90
```

```
ices
print(totals)
```

R provides useful built-in functions for common vector calculations like `sum()`, `mean()`, `min()` and `max()`:

```
scores <- c(85, 92, 78, 9
5, 88)
print(sum(scores))
print(mean(scores))
print(min(scores))
print(max(scores))
print(rev(scores))
```

```
#Output:
[1] 438
[1] 87.6
[1] 78
[1] 95
[1] 88 95 78 92 85
```

→ cumulative sum: `sum(numbers[1:i])`

Named Vectors

Named vectors let you assign labels to each element, making your code more readable and self-documenting.

You can create a named vector by providing names during creation:

```
ages <- c(alice = 25), bob = 30, charlie = 28)
print(ages)
#Output:
  alice      bob charlie
    25      30      28
```

To access elements by name, use the name as a string inside the square brackets:

```
print(ages["bob"])
#Output:
bob
  30
```

You can also add names to an existing vector using the `names()` function:

```
scores <- c(85, 92, 78)
names(scores) <- c("math", "science", "english")
print(scores["math"])
```

```
#Output:
math
85
```

Iterating over Vector Elements

While R's vectorized operations are powerful, sometimes you need to process each element individually with custom logic. A `for` loop lets you iterate through vector elements one at a time.

The simplest way to loop through a vector is to iterate directly over its elements:

```
fruits <- c("apple", "banana", "cherry")
for (i in fruits){
  print(i)
}
```

```
#Output:
[1] "apple"
[1] "banana"
[1] "cherry"
```

You can combine iteration with conditional logic to process elements selectively:

```
numbers <- c(3, 8, 2, 10, 5)
for (i in numbers){
  if(i > 5){
    print(i)
  }
}
```

```
#Output:
[1] 8
[1] 10
```

This pattern is useful when you need to apply different operations based on each element's value, something that vectorized operations alone can't always handle.

Using `seq_along()`

Sometimes you need both the element's value and its position in the vector. While iterating directly over elements is clean, it doesn't give you access to the index. The `seq_along()` function solves this by generating a sequence of indices for any vector.

The `seq_along()` function returns a sequence from 1 to the length of the vector:

```
fruits <- c("apple", "banana", "cherry")
print(seq_along(fruits))
```

```
#Output:
[1] 1 2 3
```

By looping over these indices, you can access both the position and the value:

```
fruits <- c("apple", "banana", "cherry")
for (i in seq_along(fruits)){
  cat(i, ":", fruits[i], "\n")
}
```

```
#Output:
1 : apple
2 : banana
3 : cherry
```

This approach is especially useful when you need to modify elements at specific positions or compare elements with their neighbors. Using `seq_along()` is safer than `1:length(vec)` because it handles empty vectors correctly, returning an empty sequence instead of `1:0`.

Iterating Characters (strsplit)

What if you need to process each character in a string individually? Unlike vectors, you can't directly loop through a string's characters in R. The `strsplit()` function solves this by splitting a string into a vector of characters.

The `strsplit()` function takes two arguments: the string to split and the separator. To split into individual characters, use an empty string `""` as the separator:

```
word <- "hello"
chars <- strsplit(word, "")[[1]]
```

```
#Output:
[1] "h" "e" "l" "l" "o"
```

Notice the `[[1]]` at the end. This is because `strsplit()` returns a list, and we need to extract the first element to get our character vector (essentially turns the output list into an output vector). Once you have the characters as a vector, you can iterate over them using techniques you've already learnt:

```
word <- "R"
chars <- strsplit(word,
  "")[[1]]
for(char in chars) {
  print(char)
}
```

```
#Output:
[1] "R"
```

This technique is essential for tasks like counting specific characters, reversing strings, or validating input character by character. To create a vector from list of comma separated elements, use:

```
# Create character vectors from each comma-separated input
using strsplit()
inventory <- unlist(strsplit(inventory_input, ","))
shopping_list <- unlist(strsplit(shopping_list_input, ","))
sale_items <- unlist(strsplit(sale_items_input, ","))
```

String Manipulation Functions

These functions help you measure, combine, and transform text data.

The `nchar()` function returns the number of characters in a string, including spaces:

```
word <- "hello"
print(nchar(word))
```

```
#Output:
[1] 5
```

To combine multiple strings into one, use `paste()`. By default, it separates element with a space:

```
result <- paste("Hello",
  "World")
```

```
#Output:
[1] "Hello World"
```

```
print(result)
```

Use `paste0()` when you want no separator between the strings:

```
result <- paste0("Hello",  
"World")  
print(result)
```

```
#Output:  
[1] "HelloWorld"
```

For case conversion, R provides `toupper()` and `tolower()`:

```
print(toupper("hello"))  
print(tolower("WORLD"))
```

```
#Output:  
[1] "HELLO"  
[2] "world"
```

These functions become powerful when combined with the iteration techniques, allowing you to process and transform text character by character or as whole strings.

Vector Slicing and Indexing

You can extract multiple elements at once using slicing. This technique let you grab a subset of a vector in a single operation. To extract multiple elements, pass a vector of indices inside the square brackets:

```
numbers <- c(10, 20, 30,  
40, 50)  
print(numbers[c(1, 3,  
5)])
```

```
#Output:  
[1] 20 30 50
```

You can also use the colon operator to select a range of consecutive elements:

```
numbers <- c(10, 20, 30,  
40, 50)  
print(numbers[2:4])
```

```
#Output:  
[1] 20 30 40
```

Negative indices are powerful in R. They exclude elements instead of selecting them:

```
numbers <- c(10, 20, 30,
40, 50)
print(numbers[-2])
```

```
#Output:
[1] 10 30 40 50
```

You can exclude multiple elements by passing a vector of negative indices:

```
numbers <- c(10, 20, 30,
40, 50)
print(numbers[-c(1, 5)])
```

```
[1] 20 30 40
```

You cannot mix positive and negative indices in the same selection. R will throw an error if you try `numbers[c(1, -2)]`.

Logical Indexing

Instead of specifying exact positions, you can use a logical vector to select elements. This technique, called logical indexing, lets you filter a vector based on conditions rather than positions.

When you apply a comparison to a vector, R returns a logical vector of the same length:

```
numbers <- c(10, 20, 30,
40, 50)
print(numbers > 25)
```

```
#Output:
[1] FALSE FALSE TRUE TRUE
TRUE
```

You can also use this logical vector directly inside square brackets to extract matching elements:

```
numbers <- c(10, 20, 30,
40, 50)
print(numbers[numbers > 25])
```

```
#Output:
[1] 30 40 50
```

R keeps only the elements where the logical vector is `TRUE`. This approach is much cleaner than writing loops to filter data. You can combine conditions using logical operators:

```
numbers <- c(10, 20, 30, 40, 50)
print(numbers[numbers >= 20 & numbers <= 40])
```

```
#Output:
[1] 20 30 40
```

Vector Arithmetic & Recycling

When you perform arithmetic operations between two vectors of the same length, R applies the operation element by element:

```
a <- c(1, 2, 3)
b <- c(10, 20, 30)
print(a + b)
```

```
#Output:
[1] 11 22 33
```

But what happens when the vectors have different lengths? R uses a behavior called recycling. The shorter vector is repeated to match the length of the longer one:

```
numbers <- c(1, 2, 3, 4, 5, 6)
multiplier <- c(10, 100)
print(numbers * multiplier)
```

```
#Output:
[1] 10 200 30 400 50 600
```

The %in% Operator

Sometimes you need to check whether values exist within a vector. The `%in%` operator provides a clean way to test for membership, returning `TRUE` for each element that matches and `FALSE` otherwise.

The basic syntax checks if values on the left exist in the vector on the right:

```
fruits <- c("apple", "banana", "cherry")
print("banana" %in% fruits)
```

```
#Output:
[1] TRUE
```

You can also check multiple values at once. The operator returns a logical vector with one result for each value being tested:

```
fruits <- c("apple", "banana",  
"cherry")  
check <- c("banana", "grape", "a  
pple")  
print(check %in% fruits)
```

```
#Output:  
[1] TRUE FALSE T  
RUE
```

This makes `%in%` perfect for filtering. Combined with logical indexing, you can extract elements that belong to a specific set:

```
numbers <- c(1, 5, 3, 8,  
2, 9)  
allowed <- c(1, 2, 3)  
print(numbers[numbers %i  
n% allowed])
```

```
#Output:  
[1] 1 3 2
```

The `%in%` operator is especially useful when validating user input or filtering data based on a predefined list of acceptable values.

Lists in R

Creating Lists

Vectors are powerful, but they have one limitation: all elements must be of the same type. When you need to store different types of data together, R provides lists.

A list is a flexible data structure that can hold elements of different types, including numbers, characters, logical values, and even other vectors or lists. You create a list using the `list()` function:

```
my_list <- list(42, "hell  
o", TRUE)  
print(my_list)
```

```
#Output:  
[[1]]  
[1] 42  
  
[[2]]
```

Notice how each element is displayed with its index in double brackets. This is different from vectors, where elements appear on a single line.

```
[1] "hello"

[[3]]
[1] TRUE
```

Lists can also contain vectors as elements:

```
my_list <- list(c(1, 2, 3), "text",
c(TRUE, FALSE))
print(my_list)
```

This ability to mix different data types makes lists ideal for representing complex data structures, like a person's profile containing their age, name, and membership status all in one object.

```
#Output:
[[1]]
[1] 1 2 3

[[2]]
[1] "text"

[[3]]
[1] TRUE FALSE
```

Accessing List Elements

Lists use a different syntax from accessing elements compared to vectors. The key difference is between single brackets `[]` and double brackets `[[[]]`.

Double brackets extract the actual element from the list:

```
my_list <- list(42, "hello", TRUE)
print(my_list[[1]])
```

```
#Output:
[1] 42
```

Single brackets return a sublist containing the element, not the element itself:

```
my_list <- list(42, "hello", TRUE)
print(my_list[1])
```

```
#Output:
[[1]]
[1] 42
```

This distinction matters when you want to work with the actual value. If a list element is a vector, double brackets let you access it directly and then use

vector indexing:

```
my_list <- list(c(10, 20, 30), "text")
print(my_list[[1]][2])
```

```
#Output:
[1] 20
```

Here, `my_list[[1]]` retrieves the vector, and `[2]` accesses its second element. Remember: use `[[ ]]` when you need the actual content, and `[ ]` when you want a smaller list.

Modifying Lists

You can change existing elements, add new ones, or remove elements entirely. To change an existing element, use double brackets with assignment:

```
my_list <- list(10, "hello", TRUE)
my_list[[2]] <- "world"
print(my_list)
```

```
#Output:
```

```
[[1]]
[1] 10
```

```
[[2]]
[1] "world"
```

```
[[3]]
[1] TRUE
```

To add a new element, simply assign an index that doesn't exist yet:

```
my_list <- list(10, "hello")
my_list[[3]] <- FALSE
print(my_list)
```

```
#Output:
```

```
[[1]]
[1] 10
```

```
[[2]]  
[1] "hello"  
  
[[3]]  
[1] FALSE
```

To remove an element, assign `NULL` to it:

```
my_list <- list(10, "hello", TRUE)  
my_list[[2]] <- NULL  
print(my_list)
```

```
#Output:  
[[1]]  
[1] 10  
  
[[2]]  
[1] TRUE
```

Notice that after removing the second element, the remaining elements shift to fill the gap. The element that was at position 3 is now at position 2.

Named Lists

Just like named vectors, lists can have names assigned to their elements. This makes your code more readable and allows you to access elements by meaningful labels instead of numeric positions.

You can assign names when creating the list:

```
person <- list(name = "Alice", age = 25, active = TRUE)  
print(person)  
  
#Output:  
$name  
[1] "Alice"  
  
$age  
[1] 25
```

```
$active  
[1] TRUE
```

Notice how the output shows `$name` instead of `[[1]]`. To access named elements, use the `$` operator:

```
person <- list(name = "Alice", age = 25,  
active = TRUE)  
print(person$age)
```

```
#Output:  
[1] 25
```

You can also use double brackets with the name as a string, which is useful when the name is stored in a variable:

```
person <- list(name = "Alice", age = 25)  
print(person[["name"]])
```

```
#Output:  
[1] "Alice"
```

Modifying named elements works the same way. Use `$` to change existing values or add new ones:

```
person <- list(name = "Alice", age = 25)  
person$age <- 26  
person$city <- "Paris"  
print(person)
```

```
#Output:  
$name  
[1] "Alice"  
  
$age  
[1] 26  
  
$city  
[1] "Paris"
```